

Práctica de Laboratorio 1

Lectura de Imágenes, OpenCV y NumPy

Carlos

4 de septiembre de 2026

Reglas de la práctica:

- Las preguntas son bienvenidas en cualquier momento — interrúmpanme
- Trabajo individual, pero pueden discutir en voz baja con su compañero
- Tolerancia de 10 minutos para el ingreso — después, se integran en la siguiente pausa
- Al final deben subir su notebook con las celdas ejecutadas

Objetivos de la Práctica

Al finalizar esta práctica de 90 minutos, el estudiante será capaz de:

- Leer, visualizar y guardar imágenes usando `OpenCV (cv2)`
- Comprender la representación de una imagen como arreglo `NumPy`
- Inspeccionar propiedades básicas: forma, tipo de dato, canales
- Realizar indexado, *slicing* y extracción de regiones de interés (ROI)
- Convertir entre espacios de color (BGR, RGB, escala de grises, HSV)
- Calcular estadísticas descriptivas y el histograma de una imagen

Agenda (90 minutos)

Bloque	Duración
Bienvenida y objetivos	5 min
Configuración del entorno	10 min
Lectura y visualización de imágenes (cv2)	20 min
La imagen como arreglo NumPy	20 min
Espacios de color y conversiones	15 min
Características y estadísticas de la imagen	10 min
Ejercicio integrador y cierre	10 min
Total	90 min



Configuración del Entorno (10 min)

- Se recomienda un entorno virtual dedicado
- Paquetes necesarios: `opencv-python`, `numpy`, `matplotlib`

```
stlistingstlisting
print(
    "OpenCV version:", cv2.__version__)
print("NumPy version:", np.__version__)
```

Leer una Imagen con cv2.imread

```
img = cv2.imread("gato.jpg") # BGR, uint8
img_gray = cv2.imread("gato.jpg", cv2.IMREAD_GRAYSCALE)
img_alpha = cv2.imread("gato.png", cv2.IMREAD_UNCHANGED)
```

Importante

- cv2.imread devuelve None si el archivo no existe — **siempre verificar**
- OpenCV carga en orden de canales **BGR**, no RGB
- Flags: IMREAD_COLOR (default), IMREAD_GRAYSCALE, IMREAD_UNCHANGED

```
if img is None:
    raise FileNotFoundError("No se pudo leer la imagen")
```

Visualizar una Imagen

Opción A — ventana nativa de OpenCV

```
cv2.imshow("Imagen", img)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Opción B — Matplotlib (recomendado en notebooks)

```
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
plt.imshow(img_rgb)
plt.axis("off")
plt.title("Mi imagen")
plt.show()
```

Nota: si se usa `plt.imshow` directamente sobre `img` (BGR) sin convertir, ¡los colores se ven invertidos!

Guardar una Imagen

```
cv2.imwrite("salida.png", img)
cv2.imwrite("salida_gris.jpg", img_gray, [cv2.IMWRITE_JPEG_QUALITY, 90])
```

- El formato de salida se infiere de la extensión del archivo
- `cv2.imwrite` espera el arreglo en el mismo orden de canales con que trabaja OpenCV (BGR)

Una Imagen Es un Arreglo NumPy

```
print(type(img)) # <class 'numpy.ndarray'>
print(img.shape) # (alto, ancho, canales) -> (H, W, 3)
print(img.dtype) # uint8 (valores en [0, 255])
print(img.ndim) # 3
print(img.size) # H * W * 3
```

Convención de ejes

`img.shape` → (**filas, columnas, canales**) = (alto, ancho, canales). ¡Ojo! Es lo opuesto al orden intuitivo (ancho, alto) de otras librerías.

```
H, W, C = img.shape
print(f"Alto={H}, Ancho={W}, Canales={C}")
```

Indexado de Píxeles y Canales

```
pixel = img[100, 200] # array([B, G, R]) en la fila 100, col 200
b, g, r = img[100, 200]

canal_azul = img[:, :, 0]
canal_verde = img[:, :, 1]
canal_rojo = img[:, :, 2]

print(canal_azul.shape) # (H, W)
```

- Primer índice = fila (y), segundo índice = columna (x)
- $\text{img}[y, x] \neq \text{img}[x, y]$ — error clásico de principiante

Slicing y Regiones de Interés (ROI)

```
# Recortar una region rectangular: filas 50-200, columnas 100-300
roi = img[50:200, 100:300]

# Reemplazar una region por otra
img[0:150, 0:150] = roi_pequena

# Voltear con slicing negativo
img_flip_h = img[:, ::-1] # espejo horizontal
img_flip_v = img[::-1, :] # espejo vertical

# Reducir resolucion tomando cada 2do pixel
img_sub = img[:, ::2, ::2]
```

- Un slice de NumPy es una **vista**, no una copia — usar `.copy()` si se necesita independencia

Copias vs. Vistas — Trampa Común

```
roi = img[50:200, 100:300] # vista: comparte memoria con img  
roi[:] = 0 # esto TAMBIEN modifica img !!
```

```
roi_seguro = img[50:200, 100:300].copy()  
roi_seguro[:] = 0 # esto NO modifica img
```

Regla práctica

Si vas a modificar un recorte sin afectar la imagen original, usa `.copy()` explícitamente.

Conversión entre Espacios de Color

```
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

print(gray.shape) # (H, W) -- 1 solo canal
print(hsv.shape) # (H, W, 3) -- Hue, Saturation, Value
```

- Escala de grises: $Y = 0,299R + 0,587G + 0,114B$ (aprox. usada por OpenCV)
- HSV es útil para segmentar por color (más robusto a cambios de iluminación que RGB)

Ejemplo: Segmentación Simple por Color en HSV

```
hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# Rango aproximado para tonos rojos
bajo1 = np.array([0, 100, 100]); alto1 = np.array([10, 255, 255])
bajo2 = np.array([160, 100, 100]); alto2 = np.array([179, 255, 255])

mascara = cv2.inRange(hsv, bajo1, alto1) | cv2.inRange(hsv, bajo2, alto2)
resultado = cv2.bitwise_and(img, img, mask=mascara)
```

- `cv2.inRange` devuelve una máscara binaria (uint8, valores 0 o 255)
- El rojo está en ambos extremos del círculo de matiz (Hue) en OpenCV: $[0, 10] \cup [160, 179]$

```
print("Minimo:", img.min(), " Maximo:", img.max())
print("Media:", img.mean(), " Desv. estandar:", img.std())

media_por_canal = img.mean(axis=(0, 1)) # promedio B, G, R
print("Media por canal (B,G,R):", media_por_canal)

pixeles_unicos = np.unique(gray)
print("Valores de intensidad distintos:", len(pixeles_unicos))
```

- axis=(0,1) promedia sobre filas y columnas, dejando un valor por canal
- Útil para detectar imágenes muy oscuras/claras o canales saturados

Histograma de Intensidades

```
hist = cv2.calcHist([gray], [0], None, [256], [0, 256])

plt.plot(hist)
plt.title("Histograma de intensidades")
plt.xlabel("Nivel de gris"); plt.ylabel("Frecuencia")
plt.show()

# Version equivalente solo con NumPy
hist_np, bins = np.histogram(gray.ravel(), bins=256, range=(0, 256))
```

- El histograma resume la distribución de intensidades — clave para contraste, umbralización, ecualización

```
resized = cv2.resize(img, (300, 200)) # (ancho, alto) !!  
resized_scale = cv2.resize(img, None, fx=0.5, fy=0.5)  
  
(h, w) = img.shape[:2]  
centro = (w // 2, h // 2)  
M = cv2.getRotationMatrix2D(centro, angle=45, scale=1.0)  
rotada = cv2.warpAffine(img, M, (w, h))
```

Cuidado con el orden

`cv2.resize(img, (ancho, alto))` usa **(ancho, alto)** — lo opuesto a `img.shape` que da **(alto, ancho, canales)**.

Ejercicio Integrador (10 min)

Con una imagen propia (o la proporcionada en el material del curso):

- ➊ Cargar la imagen y verificar que no sea None
- ➋ Imprimir forma, tipo de dato y número de canales
- ➌ Extraer y mostrar por separado los canales B, G, R
- ➍ Convertir a escala de grises y a HSV
- ➎ Calcular media, desviación estándar y valores min/max
- ➏ Graficar el histograma de la versión en escala de grises
- ➐ Recortar el cuadrante central de la imagen (ROI) y guardarlo como `roi.png`

- ¿Por qué OpenCV usa BGR en lugar de RGB? (razones históricas)
- ¿Qué problema surge si se le pasa una imagen `float` en $[0, 1]$ a `cv2.imshow`?
- ¿Cuándo conviene trabajar en HSV en vez de RGB/BGR?
- ¿Qué diferencia hay entre una vista y una copia en NumPy, y por qué importa aquí?

- Una imagen digital es, ante todo, un arreglo NumPy de forma (H, W, C)
- OpenCV lee/escribe en **BGR**; hay que convertir a RGB para visualizar correctamente con Matplotlib
- El indexado y slicing de NumPy permiten manipular píxeles, canales y ROIs eficientemente
- Las conversiones de espacio de color (gris, HSV) son la base de muchas tareas de visión por computadora
- Estadísticas simples (media, std, histograma) ya revelan mucho sobre una imagen

Entrega de hoy

Subir el notebook (.ipynb) con las celdas del ejercicio integrador ejecutadas y sus resultados visibles.

Próxima práctica

Filtrado espacial: convoluciones, `cv2.filter2D`, suavizado y detección de bordes.

-  OpenCV Developers, *OpenCV-Python Tutorials*. <https://docs.opencv.org/>
-  NumPy Developers, *NumPy Documentation*. <https://numpy.org/doc/>
-  G. Bradski and A. Kaehler, *Learning OpenCV: Computer Vision with the OpenCV Library*. O'Reilly Media, 2008.
-  C. R. Harris et al., “Array programming with NumPy,” *Nature*, 585, 357–362, 2020.
-  R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed. Springer, 2022.
-  J. D. Hunter, “Matplotlib: A 2D Graphics Environment,” *Computing in Science & Engineering*, 9(3), 90–95, 2007.